

Universidad del Valle de Guatemala
Curso: Programación orientada a objetos
Segundo Ciclo, 2026



Laboratorio 2

REQUISITOS FUNCIONALES

1. Solicitar al iniciar el programa el nombre del caso, su código y el nombre del detective responsable.
2. Permitir crear un nuevo caso que reemplace al anterior y comience sin ubicaciones ni pistas.
3. Almacenar un máximo de cinco ubicaciones mediante un arreglo básico de objetos.
4. Registrar para cada ubicación su código, nombre, dirección, riesgo y estado.
5. Validar que el riesgo de una ubicación esté entre 1 y 10.
6. Permitir registrar una ubicación en una posición indicada por el usuario, entre 0 y 4.
7. Verificar que la posición seleccionada esté disponible y que la ubicación sea válida.
8. Mantener en null las posiciones que no tengan una ubicación registrada.
9. Consultar todas las ubicaciones registradas, mostrando sus posiciones y omitiendo los espacios vacíos.
10. Consultar una ubicación por su posición e informar si la posición es inválida o está vacía.
11. Modificar el riesgo o el estado de una ubicación registrada.
12. Descartar una ubicación y dejar su posición en null.
13. Almacenar una cantidad variable de pistas mediante un ArrayList<Pista>, sin utilizar List.
14. Registrar para cada pista su código, descripción, tipo de evidencia, importancia y confiabilidad.
15. Validar que la importancia esté entre 1 y 10 y la confiabilidad entre 0 y 100.
16. Impedir el registro de pistas con códigos repetidos.
17. Consultar todas las pistas e informar cuando no existan registros.
18. Buscar una pista por su código recorriendo el ArrayList.

19. Modificar el código, descripción, evidencia, importancia o confiabilidad de una pista existente.
20. Impedir que al modificar el código de una pista se utilice el código de otra.
21. Eliminar una pista mediante su código.
22. Calcular la cantidad de ubicaciones registradas y los espacios disponibles.
23. Identificar la ubicación con mayor riesgo.
24. Calcular la cantidad de pistas registradas.
25. Identificar la pista con mayor importancia y la pista con mayor confiabilidad.
26. Calcular el promedio de importancia de las pistas.
27. Generar un reporte con los resultados de la investigación, considerando los casos en que no existan ubicaciones o pistas.
28. Rechazar los textos obligatorios que sean null, vacíos o contengan solamente espacios.
29. Manejar las entradas numéricas incorrectas mediante `InputMismatchException`, descartar la entrada incorrecta y solicitar nuevamente el número.
30. Manejar los valores inválidos mediante `IllegalArgumentException`.
31. Utilizar bloques try-catch para informar los errores sin finalizar inesperadamente el programa.
32. Utilizar un bloque finally para cerrar el Scanner al terminar.
33. Presentar un menú con las trece opciones indicadas en el enunciado y permitir salir del programa.
34. ¿QUÉ PROPIEDADES Y MÉTODOS TENDRÁ CADA CLASE?

CLASE UBICACION

Propósito: representar un lugar relacionado con la investigación y validar sus datos.

Propiedades:

- `codigo`: identifica la ubicación.
- `nombre`: almacena el nombre del lugar.
- `direccion`: almacena la dirección o descripción del lugar.
- `riesgo`: representa el nivel de riesgo de investigar la ubicación.
- `estado`: indica el estado de la ubicación.

Métodos:

- Ubicacion(...): construye una ubicación, valida los datos recibidos e inicializa sus propiedades.
- getCodigo(): devuelve el código.
- getNombre(): devuelve el nombre.
- getDireccion(): devuelve la dirección.
- getRiesgo(): devuelve el nivel de riesgo.
- getEstado(): devuelve el estado.
- setRiesgo(...): valida y modifica el riesgo.
- setEstado(...): valida y modifica el estado.
- toString(): devuelve un texto con los datos de la ubicación.

CLASE PISTA

Propósito: representar una evidencia encontrada durante la investigación y validar sus datos.

Propiedades:

- codigo: identifica la pista y permite buscarla.
- descripcion: explica el contenido de la pista.
- evidencia: indica el tipo de evidencia.
- importancia: representa la relevancia de la pista.
- confiabilidad: representa el nivel de confianza en la pista.

Métodos:

- Pista(...): construye una pista, valida los datos e inicializa sus propiedades.
- getCodigo(): devuelve el código.
- getDescripcion(): devuelve la descripción.
- getEvidencia(): devuelve el tipo de evidencia.
- getImportancia(): devuelve la importancia.
- getConfiabilidad(): devuelve la confiabilidad.
- setCodigo(...): valida y modifica el código. La verificación de códigos repetidos se realiza en Caso.

- setDescription(...): valida y modifica la descripción.
- setEvidencia(...): valida y modifica el tipo de evidencia.
- setImportancia(...): valida y modifica la importancia.
- setConfiabilidad(...): valida y modifica la confiabilidad.
- toString(): devuelve un texto con los datos de la pista.

CLASE CASO

Propósito: administrar los datos de una investigación, sus ubicaciones y sus pistas.

Propiedades:

- nombre: almacena el nombre de la investigación.
- codigo: identifica el caso.
- detective: almacena el nombre del detective responsable.
- ubicaciones: almacena las cinco posiciones disponibles para ubicaciones.
- pistas: almacena las pistas encontradas en una colección de tamaño variable.

Métodos:

- Caso(...): valida los datos del caso e inicializa el arreglo y el ArrayList.
- getNombre(): devuelve el nombre del caso.
- getCodigo(): devuelve el código del caso.
- getDetective(): devuelve el nombre del detective.
- registrarUbicacion(...): valida la posición, el objeto y la disponibilidad del espacio antes de guardar una ubicación.
- consultarUbicacion(...): devuelve los datos de la ubicación almacenada en una posición.
- consultarUbicaciones(): devuelve los datos y posiciones de las ubicaciones registradas; informa si no existen.
- modificarUbicacionRiesgo(...): modifica el riesgo de una ubicación existente.
- modificarUbicacionEstado(...): modifica el estado de una ubicación existente.
- descartarUbicacion(...): elimina una ubicación asignando null a su posición.
- codigoNoRepetido(...): recorre las pistas y devuelve si un código está disponible.
- registrarPista(...): agrega una pista válida si su código no está repetido.

- consultarPista(...): busca una pista por código y devuelve sus datos o un mensaje si no existe.
- consultarPistas(): devuelve los datos de todas las pistas o un mensaje si no existen.
- modificarPistacodigo(...): modifica el código de una pista, verificando que no pertenezca a otra.
- modificarPistaDescripcion(...): modifica la descripción de una pista existente.
- modificarPistaEvidencia(...): modifica el tipo de evidencia de una pista existente.
- modificarPistaImportancia(...): modifica la importancia de una pista existente.
- modificarPistaConfiabilidad(...): modifica la confiabilidad de una pista existente.
- eliminarPista(...): localiza una pista por código y la elimina.
- contarUbicaciones(): cuenta las posiciones del arreglo que no contienen null.
- ubicacionesDisponibles(): calcula cuántas posiciones están disponibles.
- obtenerUbicacionMayorRiesgo(): devuelve los datos de la ubicación con mayor riesgo, omitiendo posiciones vacías.
- contarPistas(): devuelve la cantidad de pistas almacenadas.
- obtenerPistaMayorImportancia(): devuelve los datos de la pista con mayor importancia.
- obtenerPistaMayorConfiabilidad(): devuelve los datos de la pista con mayor confiabilidad.
- calcularPromedioImportancia(...): calcula el promedio de importancia; devuelve cero si recibe una colección vacía.
- generarReporte(): devuelve un texto con las cantidades, los máximos y el promedio. Evita solicitar máximos o promedios cuando no hay registros correspondientes.

CLASE PRINCIPAL

Propósito: ejecutar el programa, solicitar datos, mostrar resultados y manejar las excepciones durante la interacción con el usuario.

Propiedades:

- scanner: permite leer las entradas del teclado.
- casoActual: mantiene la referencia al caso que se está administrando.

Métodos:

- main(...): inicia el programa, solicita el caso inicial, ejecuta el menú y maneja las excepciones.
- leerEntero(...): solicita un entero y repite la lectura cuando la entrada no es válida.
- leerTexto(...): solicita y devuelve una línea de texto.
- mostrarMenu(): muestra las trece opciones del programa.
- crearNuevoCaso(): solicita los datos del caso y reemplaza el caso actual cuando la construcción es válida.
- registrarUbicacion(): solicita los datos de una ubicación y su posición, y la registra en el caso.
- consultarUbicacion(): solicita una posición y muestra la ubicación correspondiente.
- modificarUbicacion(): permite elegir entre modificar el riesgo o el estado de una ubicación.
- descartarUbicacion(): solicita una posición y descarta la ubicación.
- registrarPista(): solicita los datos de una pista y la registra.
- buscarPista(): solicita un código y muestra el resultado de la búsqueda.
- modificarPista(): permite seleccionar y modificar una propiedad de una pista.
- eliminarPista(): solicita el código de una pista y la elimina.

Las consultas generales de ubicaciones y pistas y la presentación del reporte se realizan directamente desde el switch de main(), llamando a los métodos de Caso.

CLASE ESPACIOINVALIDO

Su nombre exacto en el código es Espacioinvalido.

Propósito: representar el error que ocurre al intentar registrar una ubicación en una posición ocupada.

No declara propiedades propias. Hereda el manejo del mensaje de RuntimeException.

Método:

- EspacioInvalido(String mensaje): construye la excepción y envía el mensaje al constructor de la clase padre.

CLASE NOENCONTRADO

Su nombre exacto en el código es Noencontrado.

Propósito: representar el error que ocurre al intentar modificar o eliminar una pista cuyo código no existe.

No declara propiedades propias. Hereda el manejo del mensaje de RuntimeException.

Método:

- Noencontrado(String mensaje): construye la excepción y envía el mensaje al constructor de la clase padre.
2. ¿QUÉ TIPO DEBEN TENER LAS PROPIEDADES Y MÉTODOS DE CADA CLASE?

En Ubicacion:

- codigo, nombre, direccion y estado son String.
- riesgo es int.
- Los getters devuelven el tipo de su propiedad.
- Los setters devuelven void.
- toString() devuelve String.

En Pista:

- codigo, descripcion y evidencia son String.
- importancia y confiabilidad son int.
- Los getters devuelven el tipo de su propiedad.
- Los setters devuelven void.

- toString() devuelve String.

En Caso:

- nombre, codigo y detective son String.
- ubicaciones es Ubicacion[].
- pistas es ArrayList<Pista>.
- Los getters de los datos del caso devuelven String.
- Los métodos de registro, modificación y eliminación devuelven void.
- codigoNoRepetido() devuelve boolean.
- Los métodos de consulta, los métodos que presentan los elementos de mayor valor y generarReporte() devuelven String.
- contarUbicaciones(), ubicacionesDisponibles() y contarPistas() devuelven int.
- calcularPromedioImportancia() devuelve double, porque el promedio puede tener decimales.

En Principal:

- scanner es Scanner.
- casoActual es Caso.
- leerEntero() devuelve int.
- leerTexto() devuelve String.
- Los demás métodos devuelven void.

Los constructores de todas las clases no tienen tipo de retorno, ni siquiera void.

3. ¿QUÉ PROPIEDAD SE IMPLEMENTARÁ CON UN ARREGLO BÁSICO?

La propiedad ubicaciones de Caso se implementará como un arreglo básico de objetos Ubicacion.

Su tamaño será fijo, de cinco posiciones:

```
private Ubicacion[] ubicaciones;
```

Se inicializará en el constructor:

```
this.ubicaciones = new Ubicacion[5];
```

Los índices válidos serán del 0 al 4. Inicialmente, todas las posiciones contendrán null.

4. ¿QUÉ PROPIEDAD SE IMPLEMENTARÁ CON UN ARRAYLIST?

La propiedad pistas de Caso se implementará mediante un ArrayList<Pista>.

```
private ArrayList<Pista> pistas;
```

Se inicializará en el constructor:

```
this.pistas = new ArrayList<>();
```

Almacenará objetos Pista. Su tamaño podrá aumentar al registrar pistas y disminuir al eliminarlas. Se utilizará ArrayList directamente, sin declarar la colección como List.

5. ¿CUÁLES SERÁN LOS MODIFICADORES DE VISIBILIDAD?

Todas las clases serán públicas.

Los atributos de Ubicacion, Pista y Caso serán privados para evitar su modificación directa desde otras clases.

Los constructores, getters, setters y operaciones de consulta y administración serán públicos.

En Caso, los métodos codigoNoRepetido() y calcularPromedioImportancia() serán privados porque se utilizan como operaciones internas.

En Principal:

- Los atributos scanner y casoActual serán private static.
- main() será public static.
- Los demás métodos serán private static.

Los constructores de EspacioInvalido y Noencontrado serán públicos.

6. ¿QUÉ PARÁMETROS REQUERIRÁN LOS MÉTODOS?

CLASE UBICACION

- Ubicacion(String codigo, String nombre, String direccion, int riesgo, String estado).
- setRiesgo(int newriesgo).
- setEstado(String newestado).
- Los getters y toString() no reciben parámetros.

CLASE PISTA

- Pista(String codigo, String descripcion, String evidencia, int importancia, int confiabilidad).
- setCodigo(String newcodigo).
- setDescripcion(String newdescripcion).
- setEvidencia(String newevidencia).
- setImportancia(int newimportancia).
- setConfiabilidad(int newconfiabilidad).
- Los getters y toString() no reciben parámetros.

CLASE CASO

- Caso(String nombre, String codigo, String detective).
- registrarUbicacion(int posicion, Ubicacion newubicacion).
- consultarUbicacion(int posicion).
- modificarUbicacionRiesgo(int posicion, int riesgo).
- modificarUbicacionEstado(int posicion, String estado).
- descartarUbicacion(int posicion).
- codigoNoRepetido(String codigo).
- registrarPista(Pista pista).
- consultarPista(String codigo).
- modificarPistacodigo(String codigoabuscar, String newcodigo).
- modificarPistaDescripcion(String codigoabuscar, String newdescripcion).
- modificarPistaEvidencia(String codigoabuscar, String newevidencia).
- modificarPistaImportancia(String codigoabuscar, int newimportancia).
- modificarPistaConfiabilidad(String codigoabuscar, int newconfiabilidad).
- eliminarPista(String codigoabuscar).

- calcularPromedioImportancia(ArrayList<Pista> pistas).
- Los demás métodos no reciben parámetros.

CLASE PRINCIPAL

- main(String[] args): recibe los argumentos de ejecución.
- leerEntero(String mensaje): recibe el mensaje que se mostrará antes de solicitar el número.
- leerTexto(String mensaje): recibe el mensaje que se mostrará antes de solicitar el texto.
- Los demás métodos no reciben parámetros; utilizan scanner y casoActual.

CLASES DE EXCEPCIONES

- EspacioInvalido(String mensaje).
- NoEncontrado(String mensaje).

7. ¿CÓMO SE INICIALIZARÁN LOS OBJETOS Y QUÉ VALORES SE VALIDARÁN?

Los objetos recibirán sus valores iniciales mediante sus constructores. Principal solicitará los datos al usuario y los utilizará para construir los objetos.

El constructor de Caso creará un arreglo con cinco posiciones vacías y un ArrayList sin pistas. Al crear otro caso, se construirá una nueva instancia con estructuras vacías.

Antes de crear o modificar objetos se verificará que:

- Los textos obligatorios no sean null, vacíos ni contengan solamente espacios.
- El riesgo de una ubicación esté entre 1 y 10.
- La importancia de una pista esté entre 1 y 10.
- La confiabilidad de una pista esté entre 0 y 100.
- La posición de una ubicación esté entre 0 y 4.
- La posición esté libre antes de registrar una ubicación.
- Exista una ubicación antes de consultarla, modificarla o descartarla.
- Los objetos que se registren no sean null.
- El código de una nueva pista no esté repetido.

- La pista que se desea modificar o eliminar exista.
- Un nuevo código no pertenezca a otra pista. Conservar el código actual estará permitido.

En `Pista.setDescripcion()` deberá validarse `newdescripcion` antes de asignarlo a `descripcion`.

8. ¿CÓMO SE DETERMINARÁ SI UNA POSICIÓN CONTIENE UNA UBICACIÓN O NULL?

Primero se validará que la posición esté dentro de los límites del arreglo. Después se comparará el contenido con `null`:

```
if (ubicaciones[posicion] == null){
    // La posición está vacía.
}
```

Si el contenido es distinto de `null`, se podrán utilizar los métodos de la ubicación.

Al recorrer el arreglo para consultar ubicaciones, contarlas o buscar la de mayor riesgo, se omitirán las posiciones vacías.

9. ¿CÓMO SE REALIZARÁN LAS OPERACIONES EN EL ARRAYLIST?

Búsqueda:

Se recorrerá el `ArrayList` mediante un ciclo y se comparará el código solicitado con el de cada pista utilizando `.equals()`. No se utilizará `==` para comparar el contenido de los códigos.

Modificación:

Se localizará la pista por su código y se llamará al setter correspondiente. Los nuevos valores se validarán antes de asignarlos. Si se modifica el código, se comprobará que no pertenezca a otra pista.

Eliminación:

Se localizará la pista por su código y se eliminará con `pistas.remove(pista)`. En la implementación actual, el método ejecuta `return` inmediatamente después de eliminarla, sin continuar el recorrido.

Si la pista no existe, la consulta devolverá un mensaje. Las operaciones de modificación y eliminación lanzarán `Noencontrado`.

10. ¿QUÉ EXCEPCIONES PUEDEN PRODUCIRSE Y DÓNDE SE MANEJARÁN?

`InputMismatchException`:

Ocurre cuando se solicita un entero con `Scanner.nextInt()` y se ingresa texto, un decimal o un valor fuera del rango de `int`.

Se manejará dentro de `leerEntero()`. El `catch` descartará la línea incorrecta mediante `scanner.nextLine()` y volverá a solicitar el dato.

`IllegalArgumentException`:

Se generará cuando se reciban textos inválidos, niveles fuera de los rangos permitidos, posiciones fuera de 0 a 4, objetos nulos para registrar o códigos repetidos.

Se manejará durante la creación inicial del caso y en el `try-catch` del menú.

`Espacioinvalido`:

Se generará al intentar registrar una ubicación en una posición ocupada. Se manejará en el `try-catch` del menú.

`Noencontrado`:

Se generará al intentar modificar o eliminar una pista inexistente. Se manejará en el `try-catch` del menú.

`NullPointerException`:

En la implementación actual se lanza explícitamente al consultar, modificar o descartar una ubicación en una posición vacía. También se lanza al solicitar un máximo cuando no existen registros correspondientes.

Se manejará en el try-catch del menú. generarReporte() comprobará previamente si hay registros para evitar estas excepciones durante la elaboración del reporte.

ArrayIndexOutOfBoundsException:

Podría producirse al acceder a una posición inexistente del arreglo. Se evitará validando la posición antes del acceso y lanzando IllegalArgumentException cuando esté fuera de los límites.

Uso de finally:

El método main() tendrá un bloque finally para ejecutar scanner.close() cuando termine el bloque principal, tanto si finaliza normalmente como si sale por una excepción.